CS6650 Homework 5

Product API on ECS, load tested and instrumented

Soham Santra

Part II: The Product API

A gap in the specification

While reading api.yaml I hit something that does not add up. POST /products/{productId}/details lists 404 Product not found as a response. But nothing in the Product section creates a product. If I take the file literally, no product ever exists, and every POST returns 404 forever.

So I had to pick a reading. Either POST creates the product when it is missing, or POST only updates products that already exist and I seed some at startup.

I went with update only. It is the only reading where I can actually produce a 404 on POST, and the assignment asks for every response code. Three products are seeded with IDs 1, 2 and 3. I have noted the assumption in the README.

Why every field is a pointer

weight has minimum 0 in the schema. Every other number has minimum 1. So a product weighing zero grams is valid.

That is a problem in Go, because 0 is also what an int32 holds when nothing was assigned to it. If I had used plain int32 with Gin's binding:"required" tag, Gin would see weight 0, decide the field was never sent, and return 400 for a request the spec says is fine. Pointers fix it. nil means the field is absent, 0 means somebody sent zero. There is a weight 0 case in my test suite to prove it works.

Status codes

GET only lists 200, 404 and 500. There is no 400 on that endpoint, so when someone asks for /products/abc I have nowhere to put it except 404. POST does list a 400, so the same bad ID returns 400 there. Same mistake, two different codes, because that is what the contract says.

I also made the body's product_id have to match the ID in the path. The spec does not say either way, but letting them disagree means a write to one ID could carry a different ID inside it, which seems worse than rejecting it.

For the 500 I added a test hook. A sku of FORCE_500 returns 500 on either endpoint. Nothing a well formed request does should cause a real 500, so this is the only honest way to demonstrate the code. It is flagged as a test hook in the README rather than pretending it is organic.

Data structure

A map[int32]Product behind a sync.RWMutex.

The map gives O(1) lookup, which is what I want for a workload that is mostly reads. The RWMutex is there because Gin runs every handler in its own goroutine. Without it, two concurrent requests are a data race on the map, and in Go that is not a wrong answer, it is a fatal error that kills the process and cannot be recovered from. I saw exactly that in Homework 3.

I used RWMutex rather than Mutex because reads massively outnumber writes here, so letting readers run concurrently is worth it. Worth being honest about one thing: in the Homework 3 benchmark RWMutex actually lost to a plain Mutex, 12.07ms against 8.48ms. That test was write heavy. This workload is the opposite, so the result does not apply.

Every response code

Code	Endpoint	How I trigger it
200	GET	Existing product, ID 1
404	GET	ID 9999, does not exist
404	GET	ID abc, unusable (no 400 defined on GET)
204	POST	Valid body, existing ID
400	POST	Malformed JSON
400	POST	Missing required field (sku)
400	POST	category_id of 0, below its minimum
400	POST	Body product_id does not match the path
204	POST	weight of 0, which the spec allows
404	POST	ID 9999, does not exist
500	POST	sku of FORCE_500, the test hook

The curl commands for all of these are in the README. I ran the same set three times: locally with go run, inside the Docker container, and against the deployed service. All three gave identical output.

```
soham@Sohams-MacBook-Air BSDS % cd ~/Desktop/BSDS/hw05
./test_endpoints.sh

==============================================
GET 200  existing product
==============================================
{"product_id":1,"sku":"ABC-123-XYZ","manufacturer":"Acme Corporation","category_id":456,"weight":1250,"some_oth
er_id":789}
HTTP 200

==============================================
GET 404  product does not exist
==============================================
{"error":"NOT_FOUND","message":"Product not found","details":"No product with ID 9999"}
HTTP 404

==============================================
GET 404  id is not a positive integer (spec defines no 400 on GET)
==============================================
{"error":"NOT_FOUND","message":"Product not found","details":"Product ID must be a positive integer"}
HTTP 404

==============================================
POST 204 valid update of an existing product
==============================================

HTTP 204

==============================================
POST 400 malformed JSON
==============================================
{"error":"INVALID_INPUT","message":"The provided input data is invalid","details":"Request body is not valid JS
ON: unexpected EOF"}
HTTP 400
```

Figure 1. All eleven response codes, served locally with go run.

```
soham@Sohams-MacBook-Air hw05 % curl -i http://localhost:8080/products/1
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Sun, 18 Oct 2026 20:45:29 GMT
Content-Length: 122

{"product_id":1,"sku":"ABC-123-XYZ","manufacturer":"Acme Corporation","category_id":456,"weight":1250,"some_oth
er_id":789}%
```

Figure 2. Headers on the 200, confirming the JSON content type.

Part III: Terraform

What the files do

File	What it does
provider.tf	Declares the AWS and Docker providers. Grabs an ECR auth token as a data source and feeds it to the Docker provider, which is what replaces the manual docker login
variables.tf	The knobs: region, repository name, service name, container port, task count, log retention
main.tf	Calls the four modules, then builds and pushes the image
outputs.tf	Prints the cluster and service names, which the public IP lookup needs
modules/network	Finds the default VPC and its subnets, creates a security group opening the container port
modules/ecr	Creates the registry
modules/logging	Creates the CloudWatch log group at /ecs/<service_name>
modules/ecs	Cluster, Fargate task definition, and the service that keeps a task alive

The thing I found most interesting is that nothing says what order to build in. main.tf hands the ECR repository URL to the ECS module, and that one reference is enough for Terraform to work out that ECR has to come first. The order falls out of the references rather than being written down anywhere.

data "aws_iam_role" "lab_role" is a lookup, not something Terraform creates. LabRole already exists and Terraform just reads its ARN. It gets passed as both roles: the execution role, which ECS uses to pull the image and set up logging before my container starts, and the task role, which my code uses while it runs.

What I changed

- Deleted src/main.go, dropped in product-service.go with its own go.mod and go.sum.
- Replaced src/Dockerfile, with one change described below.
- Changed variables.tf: region to us-east-1, repository to product-service, service name to product-api. I did this before the first apply on purpose, because changing service_name later would force Terraform to destroy and rebuild the cluster.
- Added a depends_on that was missing. module.ecs referenced module.ecr.repository_url, so it waited for the repository to exist, but nothing made it wait for the image to actually be pushed. Terraform could legally start the service before the image was there. The service keeps retrying the pull so it recovers on its own, but it is still a gap in the graph.

```
module "ecs" {
  source     = "./modules/ecs"
  depends_on = [docker_registry_image.app]
  ...
}
```

The architecture problem

My Mac is Apple Silicon, so arm64. Fargate is amd64. An arm64 image will not start there, it fails with an exec format error.

In the last assignment I fixed this with docker build --platform linux/amd64 on the command line. That does not work here, because Terraform runs the build and there is no command line to put the flag on. It has to go in the Dockerfile.

```
FROM golang:1.27 AS builder
RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o product-service .
```

```
FROM --platform=linux/amd64 alpine:latest
```

The builder stage runs natively and cross compiles, which is much faster than emulating an amd64 machine to build on. The final stage is pinned so the image itself is tagged amd64. I needed both. An amd64 binary inside an arm64 image would still have failed.

I checked it before running apply, with docker image inspect and the Architecture format string. It printed amd64.

Two things I noticed

This deploys an ECS service, not a standalone task like last week. A service is a supervisor. desired_count of 1 means if the task dies, ECS starts another one. Last week when my task died it just stayed dead.

There is no load balancer, which is why getting the public IP takes that four part command in the README, and why the IP changes if the task ever restarts. An ALB would give one stable address. It is out of scope here but it is clearly the next thing to add.

The plan also listed six subnets, one per availability zone. With one task that does nothing, but if I raised the count the tasks would spread across zones, so losing one datacentre would not take the whole thing down.

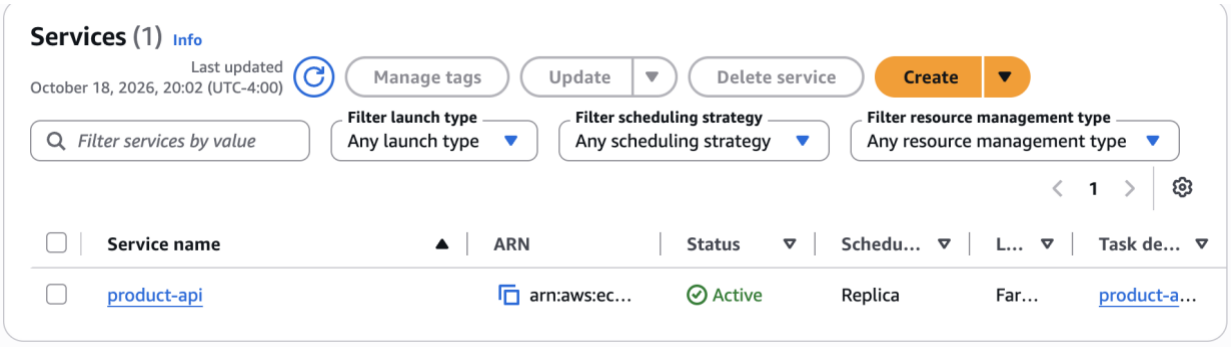


Figure 3. The product-api service on Fargate. Everything here was created by terraform apply.

```
soham@Sohams-MacBook-Air terraform % curl http://54.173.204.114:8080/health
{"status":"ok"}%
soham@Sohams-MacBook-Air terraform % ~/Desktop/BSDS/hw05/test_endpoints.sh 54.173.204.114:8080

==========================================
GET 200  existing product
==========================================
{"product_id":1,"sku":"ABC-123-XYZ","manufacturer":"Acme Corporation","category_id":456,"weight":1250,"some_oth
er_id":789}
HTTP 200

==========================================
GET 404  product does not exist
==========================================
{"error":"NOT_FOUND","message":"Product not found","details":"No product with ID 9999"}
HTTP 404

==========================================
GET 404  id is not a positive integer (spec defines no 400 on GET)
==========================================
{"error":"NOT_FOUND","message":"Product not found","details":"Product ID must be a positive integer"}
HTTP 404

==========================================
POST 204 valid update of an existing product
```

Figure 4. The same response code suite, this time against the deployed service.

Part IV: Load testing

All of these run against the deployed service at 54.173.204.114:8080, from my laptop in Boston. The locustfile has three user classes with the same request mix, so only one thing changes between runs.

How I designed the test

The mix is eight reads of a real product, one read of a product that does not exist, one write. I picked read heavy because that is what a real store looks like. People look at product pages constantly and almost never edit them.

The missing product task is there to generate errors on purpose, so the error panel in Part V has something to show. Those 404s are marked as successes in Locust, because a correct 404 is the server working properly, not failing. They still count as errors in the server's own metrics, which is what I actually wanted.

I grouped the request names as /products/[id] so that three product IDs do not get split into three separate rows in the statistics table.

Think time is between 0.1 and 0.5 seconds for the normal runs. StressUser has none at all, which turned out to matter a lot.

Results

Run	Class	Users	RPS	Median	p95	p99	Avg	Min	Max	Fails
1	ProductUser	10	29.1	29	110	120	41.82	18	260	0
2	ProductUser	50	143.6	33	110	120	44.91	18	318	0
3	ProductFastUser	50	145.1	30	110	120	42.54	18	291	0
4	StressUser	200	3,337	45	130	160	56.83	21	311	0
5	StressUser	500	4,294	96	190	240	104.82	23	412	0

Everything in milliseconds. No failures in any run, including 513,526 requests at 500 users.

I also ran runs 2 and 3 again headless with --csv for exactly two minutes each, so the client comparison would be properly controlled. Those numbers are in the repo and match the UI runs to within two percent.

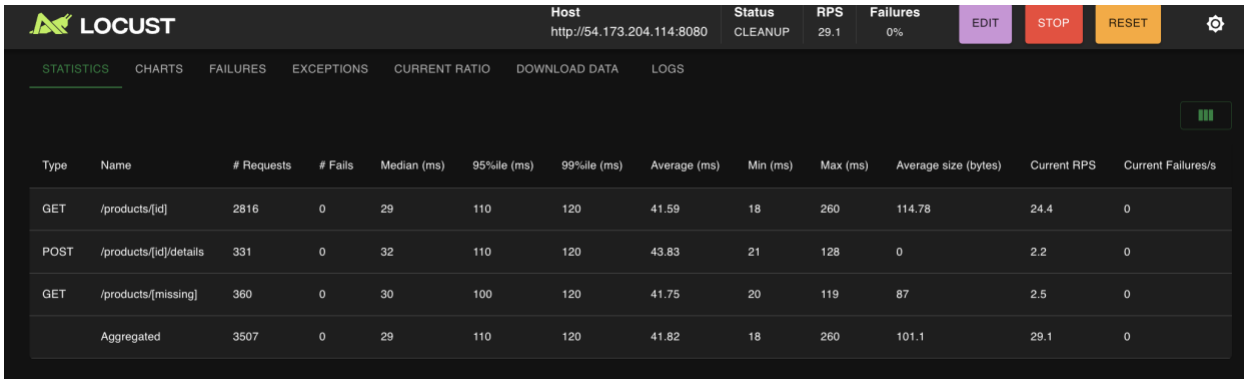
The server was doing nothing for the first two runs

Runs 1 and 2 scale almost exactly linearly. Five times the users, 4.9 times the throughput, and the latency barely moves. Median goes 29 to 33ms and p95 does not move at all.

That is not the server working harder. Every request is just sitting on the network. The min is 18ms, which is about what a round trip to Virginia costs, and my handlers finish in well under a millisecond, so almost everything I measured was travel time.

The throughput was my own load generator's arithmetic. Fifty users divided by roughly 0.33 seconds per cycle is about 150 requests per second, and I measured 143.6. The client was the bottleneck, not the server, and adding more users with think time would not have changed that.

This is the same thing that caught me out in Homework 3, where four Locust workers gave zero extra throughput. That is why StressUser has wait_time set to constant(0). Taking the think time out, with nothing else changed, took throughput from 143 to 3,337 requests per second against the same server.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	2816	0	29	110	120	41.59	18	260	114.78	24.4	0
POST	/products/[id]/details	331	0	32	110	120	43.83	21	128	0	2.2	0
GET	/products/[missing]	360	0	30	100	120	41.75	20	119	87	2.5	0
	Aggregated	3507	0	29	110	120	41.82	18	260	101.1	29.1	0

Figure 5. Run 1, ten users.

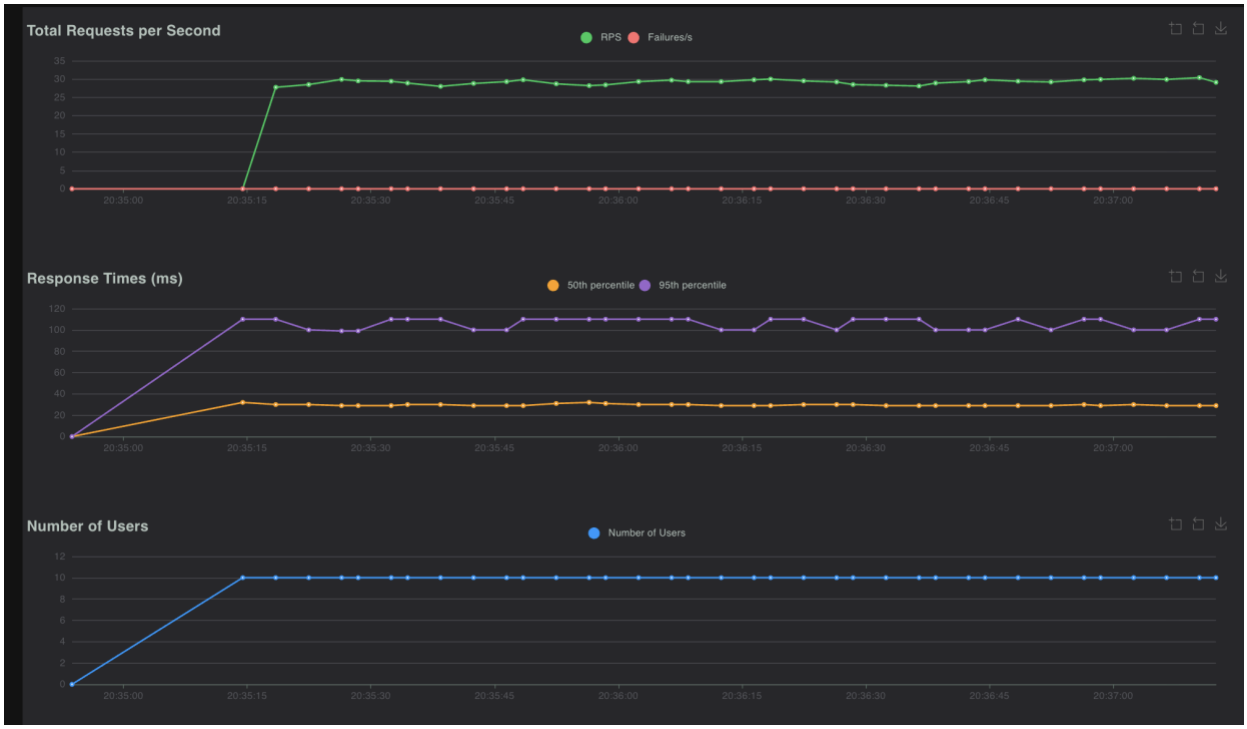
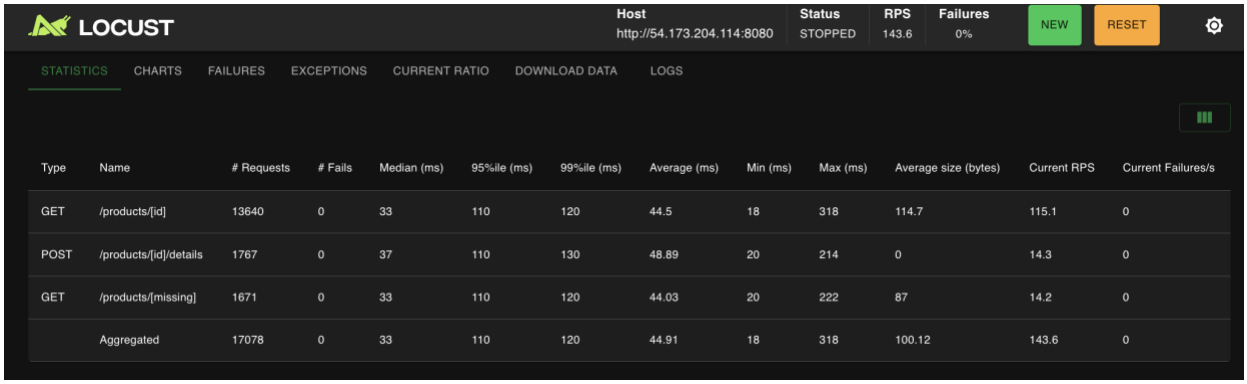


Figure 6. Run 1 charts. Everything flat: RPS, both percentiles, and zero failures.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	13640	0	33	110	120	44.5	18	318	114.7	115.1	0
POST	/products/[id]/details	1767	0	37	110	130	48.89	20	214	0	14.3	0
GET	/products/[missing]	1671	0	33	110	120	44.03	20	222	87	14.2	0
	Aggregated	17078	0	33	110	120	44.91	18	318	100.12	143.6	0

Figure 7. Run 2, fifty users. Five times the load, same latency.



Figure 8. Run 2 charts. The p95 spikes to 140ms during the ramp, then settles at 110ms.

HttpUser against FastHttpUser

Runs 2 and 3 are the same load with a different HTTP client underneath.

Metric	HttpUser	FastHttpUser	Difference
RPS	143.6	145.1	+1%
Median	33ms	30ms	-3ms
Average	44.91ms	42.54ms	-2.4ms
p95	110ms	110ms	none
p99	120ms	120ms	none
Min	18ms	18ms	none

Throughput is the same. Median and average come down by about five percent. The percentiles do not move at all.

HttpUser uses python-requests, which does more work per request and has to fight the GIL. FastHttpUser uses geventhttpclient, which is lighter. The 2 to 3ms gap is the real cost of that difference.

What I find interesting is that I ran this exact comparison in Homework 3 against localhost and got a much bigger result there, p99 down 39 percent and max down 71 percent. Nothing about the two clients changed. What changed is everything else in the measurement.

On localhost the round trip was basically zero, so the client's 2 to 3ms was a big share of the total and taking it away was obvious. Here the round trip is about 30ms, so the same 2 to 3ms is roughly five percent of the total and you can barely see it.

This is Amdahl's law. You cannot speed something up by more than the share of it you are speeding up. The same change looked like a big win in one setup and noise in the other, and the law explains both.

One place it did show up is the ramp. Run 2's p95 overshoots to 140ms before settling, and run 3 never overshoots. Connection setup under a burst of new users is exactly where a lighter client helps.

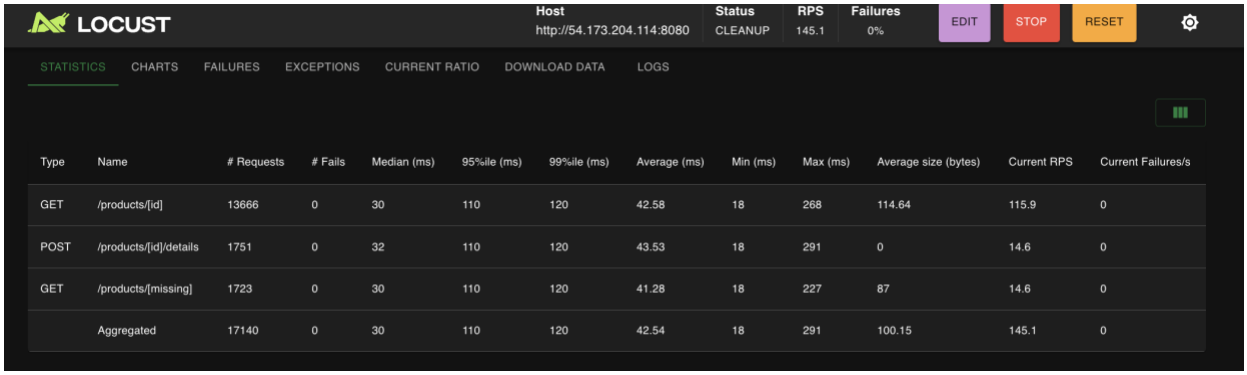


Figure 9. Run 3, fifty users on FastHttpUser.

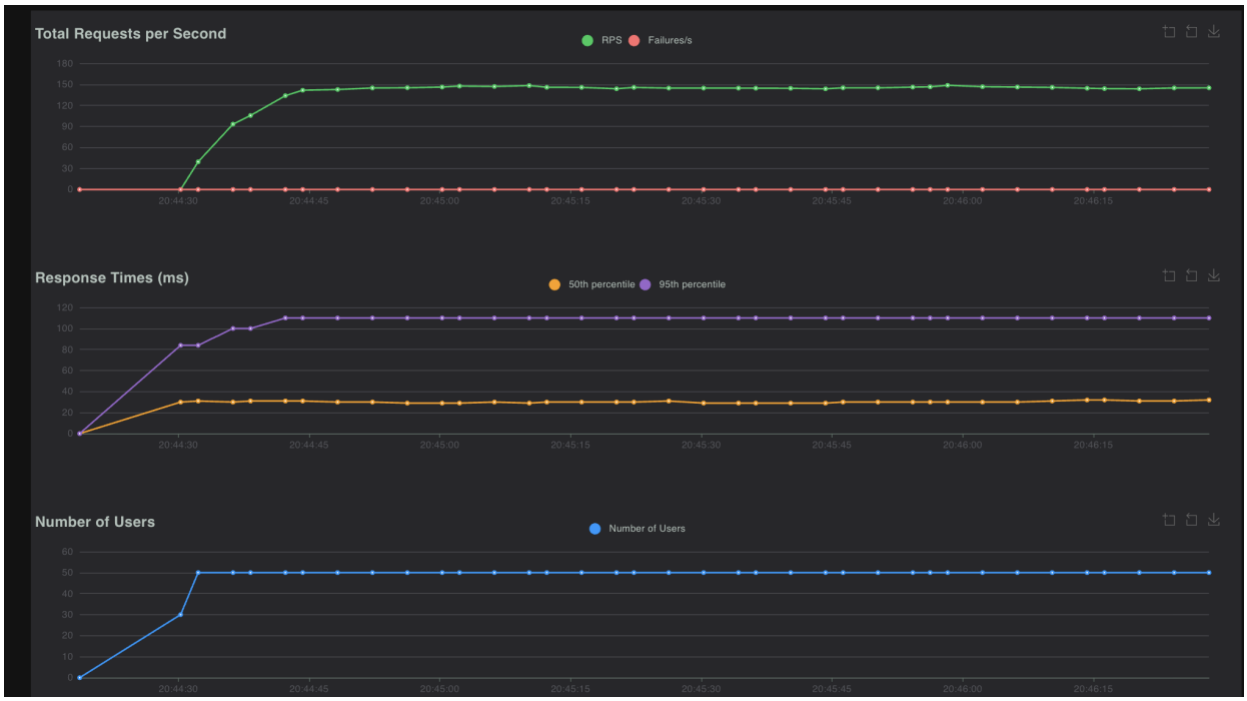


Figure 10. Run 3 charts. No p95 overshoot on the ramp, unlike run 2.

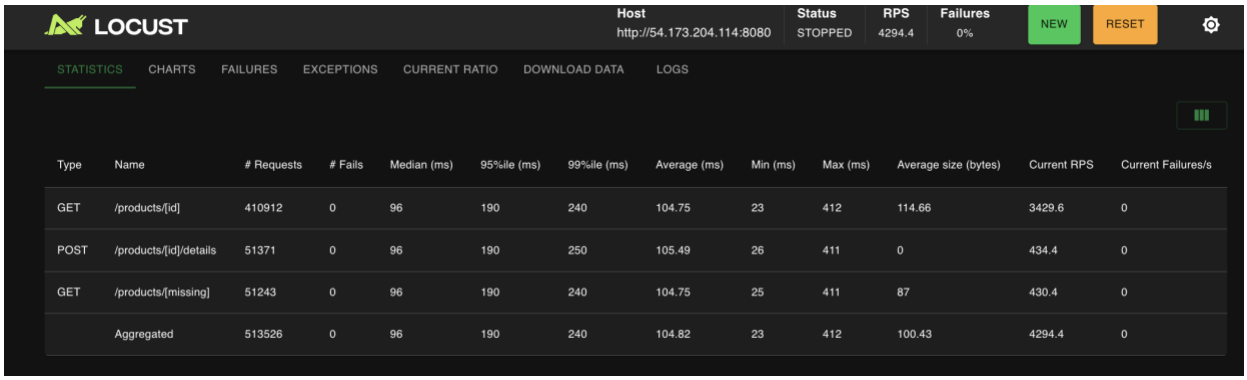
Where it started to struggle

Metric	200 users	500 users	Ratio
RPS	3,337	4,294	1.29x
Median	45ms	96ms	2.13x
p95	130ms	190ms	1.46x
Min	21ms	23ms	1.10x

2.5 times the load bought me 1.29 times the throughput and cost 2.13 times the latency. Past about 200 users I am buying queue, not work. Throughput per user drops from 16.7 to 8.5 requests per second, which is almost exactly half. The ceiling is somewhere around 4,300.

The min went up too, 21ms to 23ms. Even the fastest request got slower, so the queue was never empty.

I expected the charts within a single run to show latency climbing, and they do not, they go flat. That turns out to be correct. With a fixed number of users and no think time, throughput times latency equals the user count, so the whole thing settles at an equilibrium. The knee shows up when you compare runs, not inside one.



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/products/[id]	410912	0	96	190	240	104.75	23	412	114.66	3429.6	0
POST	/products/[id]/details	51371	0	96	190	250	105.49	26	411	0	434.4	0
GET	/products/[missing]	51243	0	96	190	240	104.75	25	411	87	430.4	0
	Aggregated	513526	0	96	190	240	104.82	23	412	100.43	4294.4	0

Figure 11. Run 5, five hundred users, no think time. 513,526 requests and still zero failures.



Figure 12. Run 5 charts. RPS plateaus around 4,300 and latency sits at a new, higher level.

Checking which side actually ran out

A slowdown on its own does not tell you whose fault it is, so I checked both ends during run 5.

My laptop was fine. top showed Locust at 37.4 percent of one core and the machine 87 percent idle. So the generator was not the limit this time, which is the opposite of what happened in Homework 3.

The server was the limit. CloudWatch for the ECS service:

Time (EDT)	Average CPU	Max CPU
20:49	0.005%	0.01%
20:50	0.02%	0.07%
20:51	33.15%	99.46%
20:52	99.49%	99.83%
20:53	66.10%	99.33%
20:54	0.0%	0.0%

The task has cpu set to 256, which is a quarter of one vCPU. It sat at 99.49 percent for the entire middle minute of the run and dropped straight back to zero afterwards. The timestamps line up with the Locust charts.

At roughly 4,300 requests per second on a quarter core, each request is costing about 60 microseconds of CPU. That matches what I saw in the local container logs, where handlers finished in 0.015 to 0.17ms.

GetMetricStatistics

Average	Maximum	Timestamp	Unit
0.004980381578207016	0.014941144734621048	2026-10-18T20:49:00-04:00	Percent
0.021726946036020916	0.06518083810806274	2026-10-18T20:50:00-04:00	Percent
33.15204366048177	99.45613098144531	2026-10-18T20:51:00-04:00	Percent
99.49259694417317	99.82842254638672	2026-10-18T20:52:00-04:00	Percent
66.10457865397136	99.33415985107422	2026-10-18T20:53:00-04:00	Percent
0.0	0.0	2026-10-18T20:54:00-04:00	Percent
0.0	0.0	2026-10-18T20:55:00-04:00	Percent
0.0	0.0	2026-10-18T20:56:00-04:00	Percent

Figure 13. CPU for the ECS service across run 5.

Tradeoffs

Reads dominate in a product catalogue, and that drives most of the design.

- Map instead of a slice, so a lookup by ID is O(1) rather than a scan.
- RWMutex instead of Mutex, so readers do not block each other.
- Caching is the obvious next step. Product data hardly ever changes and nobody minds if it is a few seconds stale.
- Scaling out does not work as the code stands. Setting ecs_count to 2 would double the CPU and reads would be fine, but each task has its own map in memory, so a POST to one task would be invisible to the other.

That last point is the same failure I wrote up in Homework 2 Part IV, where I posted an album to one EC2 instance and the other one kept returning the old list. Same bug, different assignment.

So the CPU ceiling is a one line fix and the state problem is not. The load test found the easy limit, and thinking about the fix found the real one.

Part V: Observability

The server exposes /metrics for Prometheus and writes one JSON line per request. Both come out of the same middleware and the same timer, so the dashboard and the logs cannot end up disagreeing with each other.

Prometheus and Grafana run locally with docker compose, as the assignment says to. Grafana gets its datasource and dashboard provisioned from files, so the stack works as soon as it comes up.

The three signals

Signal	Metric	Query
Rate	http_requests_total	sum by (route) (rate(http_requests_total[1m]))
Errors	http_request_errors_total	100 * sum(rate(http_request_errors_total[1m])) / sum(rate(http_requests_total[1m]))
Duration	http_request_duration_seconds	histogram_quantile(0.95, sum by (le, route) (rate(http_request_duration_seconds_bucket[1m])))

Two things I had to get right. The metrics are labelled by route, not by path. c.FullPath() gives /products/:productId, so a million product IDs collapse into one series. If I had labelled by the raw path, every new ID would create a new time series and eventually take Prometheus down.

The histogram needed custom buckets. Prometheus defaults start at 5ms, and my handlers finish in tens of microseconds, so every single request would have landed in the first bucket and every percentile would have been meaningless. I used ExponentialBuckets(0.0001, 2, 14), which runs from 100 microseconds to about 0.8 seconds.

The dashboard

I let it sit idle, ran fifty users for three minutes, then let it go idle again, so the whole shape is in one screenshot.

The rate panel reads about 145 requests per second and Locust reported 143.6. Those are two completely separate measurements, one from the client and one from inside the server, and they agree. That cross check is the main thing instrumenting buys you.

The error panel sits at a flat 10 percent, all 404s. That is on purpose, since one request in ten asks for a product that does not exist. It is a useful reminder that 10 percent errors is normal for this service, so an alert on any error at all would be useless here. You alert on a change from the baseline, not on errors existing.

The thing I did not expect is that the slowest endpoint on the whole server is /metrics itself. My actual product routes sit on the floor at sub millisecond. The p95 for /metrics is around 12ms when idle and jumps to about 37ms under load. Under load there are more histogram buckets and more label combinations filled in, so each scrape has more to serialise. Watching the system costs more than running it, which is a decent argument for not getting carried away with labels.



Figure 14. The RED dashboard. Idle, then loaded, then idle again.

The same thing in logs

```
{"time":"2026-10-19T00:19:41.493696096Z","method":"GET",
 "route":"/products/:productId","path":"/products/1",
 "status":200,"duration_ms":0.017,"client_ip":"192.168.65.1"}

{"time":"2026-10-19T00:19:41.737771929Z","method":"GET",
 "route":"/products/:productId","path":"/products/9999",
 "status":404,"duration_ms":0.168,"client_ip":"192.168.65.1"}

{"time":"2026-10-19T00:19:41.533715471Z","method":"POST",
 "route":"/products/:productId/details","path":"/products/1/details",
 "status":204,"duration_ms":0.145,"client_ip":"192.168.65.1"}
```

Every line has the route, the status and the duration, so it is the same three signals one request at a time. I log both route and path, because the route is what you group by and the path is what you actually need when debugging.

The excerpt caught something I was not looking for. The first ten lines run from 2.9 to 23.4ms and they are all within 5ms of each other, then from line eleven onwards everything sits near 0.1ms. That is a drop of roughly 200 times. It is the cold start, the first wave of requests hitting a server that has never served one. Good reminder that the first measurement is never the real one.

The same logs are available from the deployed service through the CloudWatch log group the logging module creates, with aws logs tail /ecs/product-api.

When I would look at which

The dashboard tells me whether something is wrong and roughly when it started. It is aggregated, it is always running, and it is what an alert would point me at. What it cannot tell me is which request failed.

The logs tell me what happened to a specific request. So once the dashboard says p95 doubled at 20:52, the logs are where I go to find out whether it was one slow route, one client hammering something, or everything at once.

Dashboard first to notice and narrow it down, logs second to actually diagnose it. The dashboard is also the only one of the two that is any use before I know something is wrong.